# Industry Task 2 - Food Ordering and Billing System

Course: Programming in Java (CSA09) — Java AWT, Event Handling and Layout Managers

# Task 2: Online Food Ordering and Billing System

## Problem Statement

A restaurant wants a simple Food Ordering and Billing Application for counter staff, allowing customers to select food items, specify quantities, and generate a final bill — implemented as a multi-screen AWT POS system.

## Components and Layout Manager Used

- **Layout:** `CardLayout` on the main panel, switching between three cards: **Customer/Order Details**, **Food Selection**, **Bill/Order Summary**. Each card internally uses `GridLayout`/`FlowLayout`.
- **Components:** `TextField` (Customer Name, Table Number, Quantity), `Choice` (Food Category, Food Item), `List`/`TextArea` (order list, bill), `Button` (Add Item, Generate Bill, Clear, Next, Previous).
- **Event Handling:** `ActionListener` for every button and for the category `Choice` (to repopulate the item `Choice`).

## Java Source Code

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;

public class FoodOrderingSystem extends Frame implements ActionListener, ItemListener {

    CardLayout cardLayout = new CardLayout();
    Panel mainPanel = new Panel(cardLayout);

    // Card 1: Customer details
    TextField tfCustomer, tfTable;

    // Card 2: Food selection
    Choice choiceCategory, choiceItem;
    TextField tfQty;
    java.awt.List orderList;
    Button btnAdd, btnClearOrder;

    // Card 3: Bill summary
    TextArea billArea;

    Button btnNext1, btnNext2, btnPrev2, btnPrev3, btnGenerateBill, btnClearAll;

    // Simple menu: category -> items -> price
    LinkedHashMap<String, LinkedHashMap<String, Double>> menu = new LinkedHashMap<>();
    ArrayList<String> orderItems = new ArrayList<>();
    ArrayList<Double> orderPrices = new ArrayList<>();
    ArrayList<Integer> orderQtys = new ArrayList<>();

    public FoodOrderingSystem() {
        setTitle("Restaurant Food Ordering & Billing System");
        buildMenu();
        setLayout(new BorderLayout());

        mainPanel.add(buildCustomerCard(), "CUSTOMER");
        mainPanel.add(buildFoodCard(), "FOOD");
        mainPanel.add(buildBillCard(), "BILL");
        add(mainPanel, BorderLayout.CENTER);

        setSize(520, 480);
        setVisible(true);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });
    }

    private void buildMenu() {
        LinkedHashMap<String, Double> starters = new LinkedHashMap<>();
        starters.put("Spring Roll", 120.0);
        starters.put("Paneer Tikka", 180.0);
        menu.put("Starters", starters);

        LinkedHashMap<String, Double> mains = new LinkedHashMap<>();
        mains.put("Veg Biryani", 220.0);
        mains.put("Butter Chicken", 280.0);
        menu.put("Main Course", mains);
```

```
        LinkedHashMap<String, Double> beverages = new LinkedHashMap<>();
        beverages.put("Masala Chai", 40.0);
        beverages.put("Fresh Lime Soda", 60.0);
        menu.put("Beverages", beverages);
    }

    private Panel buildCustomerCard() {
        Panel p = new Panel(new GridLayout(4, 2, 8, 8));
        p.add(new Label("Customer Name:"));
        tfCustomer = new TextField();
        p.add(tfCustomer);

        p.add(new Label("Table Number:"));
        tfTable = new TextField();
        p.add(tfTable);

        btnNext1 = new Button("Next >>");
        btnNext1.addActionListener(this);
        p.add(new Label());
        p.add(btnNext1);
        return p;
    }

    private Panel buildFoodCard() {
        Panel p = new Panel(new BorderLayout(8, 8));

        Panel top = new Panel(new GridLayout(3, 2, 8, 8));
        top.add(new Label("Food Category:"));
        choiceCategory = new Choice();
        for (String cat : menu.keySet()) choiceCategory.add(cat);
        choiceCategory.addItemListener(this);
        top.add(choiceCategory);

        top.add(new Label("Food Item:"));
        choiceItem = new Choice();
        top.add(choiceItem);

        top.add(new Label("Quantity:"));
        tfQty = new TextField("1");
        top.add(tfQty);
        p.add(top, BorderLayout.NORTH);

        orderList = new java.awt.List(8);
        p.add(orderList, BorderLayout.CENTER);

        Panel btns = new Panel(new FlowLayout());
        btnAdd = new Button("Add Item");
        btnClearOrder = new Button("Clear");
        btnPrev2 = new Button("<< Previous");
        btnNext2 = new Button("Next >>");
        btnAdd.addActionListener(this);
        btnClearOrder.addActionListener(this);
        btnPrev2.addActionListener(this);
        btnNext2.addActionListener(this);
        btns.add(btnAdd);
        btns.add(btnClearOrder);
        btns.add(btnPrev2);
        btns.add(btnNext2);
        p.add(btns, BorderLayout.SOUTH);

        populateItems();
        return p;
    }

    private Panel buildBillCard() {
        Panel p = new Panel(new BorderLayout(8, 8));
        billArea = new TextArea(15, 45);
        billArea.setEditable(false);
        p.add(billArea, BorderLayout.CENTER);

        Panel btns = new Panel(new FlowLayout());
        btnGenerateBill = new Button("Generate Bill");
        btnPrev3 = new Button("<< Previous");
        btnClearAll = new Button("Clear / New Order");
        btnGenerateBill.addActionListener(this);
        btnPrev3.addActionListener(this);
        btnClearAll.addActionListener(this);
        btns.add(btnGenerateBill);
        btns.add(btnPrev3);
        btns.add(btnClearAll);
        p.add(btns, BorderLayout.SOUTH);
        return p;
    }

    private void populateItems() {
        choiceItem.removeAll();
        LinkedHashMap<String, Double> items = menu.get(choiceCategory.getSelectedItem());
        for (String item : items.keySet()) {
            choiceItem.add(item + " (Rs." + items.get(item) + ")");
        }
    }

    public void itemStateChanged(ItemEvent e) {
```

```
        if (e.getSource() == choiceCategory) populateItems();
    }

    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();

        if (src == btnNext1) {
            if (tfCustomer.getText().trim().isEmpty() || tfTable.getText().trim().isEmpty()) {
                return; // simple guard; a MessageDialog could be shown here
            }
            cardLayout.show(mainPanel, "FOOD");

        } else if (src == btnNext2) {
            cardLayout.show(mainPanel, "BILL");

        } else if (src == btnPrev2) {
            cardLayout.show(mainPanel, "CUSTOMER");

        } else if (src == btnPrev3) {
            cardLayout.show(mainPanel, "FOOD");

        } else if (src == btnAdd) {
            addItemToOrder();

        } else if (src == btnClearOrder) {
            orderList.removeAll();
            orderItems.clear();
            orderPrices.clear();
            orderQtys.clear();

        } else if (src == btnGenerateBill) {
            generateBill();

        } else if (src == btnClearAll) {
            resetAll();
        }
    }

    private void addItemToOrder() {
        String itemLabel = choiceItem.getSelectedItem();
        if (itemLabel == null || itemLabel.isEmpty()) return; // must select an item

        int qty;
        try {
            qty = Integer.parseInt(tfQty.getText().trim());
        } catch (NumberFormatException ex) {
            orderList.add("Invalid quantity - please enter a number");
            return;
        }
        if (qty <= 0) {
            orderList.add("Invalid quantity - must be greater than 0");
            return;
        }

        String itemName = itemLabel.substring(0, itemLabel.indexOf(" (Rs."));
        double price = menu.get(choiceCategory.getSelectedItem()).get(itemName);

        orderItems.add(itemName);
        orderPrices.add(price);
        orderQtys.add(qty);

        double lineTotal = price * qty;
        orderList.add(itemName + " x " + qty + " = Rs." + String.format("%.2f", lineTotal));
    }

    private void generateBill() {
        double subtotal = 0;
        StringBuilder sb = new StringBuilder();
        sb.append("========= BILL SUMMARY =========\n");
        sb.append("Customer : ").append(tfCustomer.getText()).append("\n");
        sb.append("Table No : ").append(tfTable.getText()).append("\n");
        sb.append("---------------------------------\n");

        for (int i = 0; i < orderItems.size(); i++) {
            double lineTotal = orderPrices.get(i) * orderQtys.get(i);
            subtotal += lineTotal;
            sb.append(String.format("%-20s x%-3d Rs.%8.2f\n",
                    orderItems.get(i), orderQtys.get(i), lineTotal));
        }

        double gst = subtotal * 0.05;
        double finalAmount = subtotal + gst;

        sb.append("---------------------------------\n");
        sb.append(String.format("Subtotal        : Rs.%.2f\n", subtotal));
        sb.append(String.format("GST (5%%)        : Rs.%.2f\n", gst));
        sb.append(String.format("FINAL AMOUNT    : Rs.%.2f\n", finalAmount));
        sb.append("=================================\n");

        billArea.setText(sb.toString());
    }

    private void resetAll() {
```

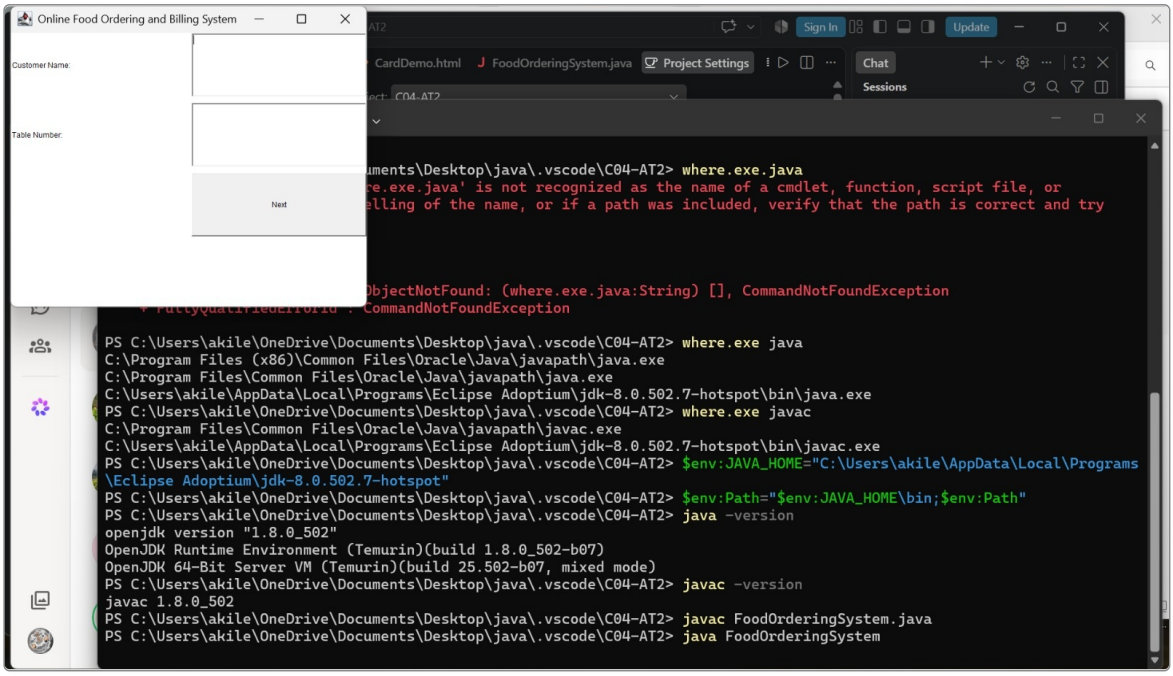
```
            tfCustomer.setText("");
            tfTable.setText("");
            tfQty.setText("1");
            orderList.removeAll();
            orderItems.clear();
            orderPrices.clear();
            orderQtys.clear();
            billArea.setText("");
            cardLayout.show(mainPanel, "CUSTOMER");
        }

        public static void main(String[] args) {
            new FoodOrderingSystem();
        }
    }
```

## Sample Order and Generated Bill

**Order entered:** Customer = `Rohit Sharma`, Table = 7; adds `Veg Biryani x 2`, `Masala Chai x 3`.

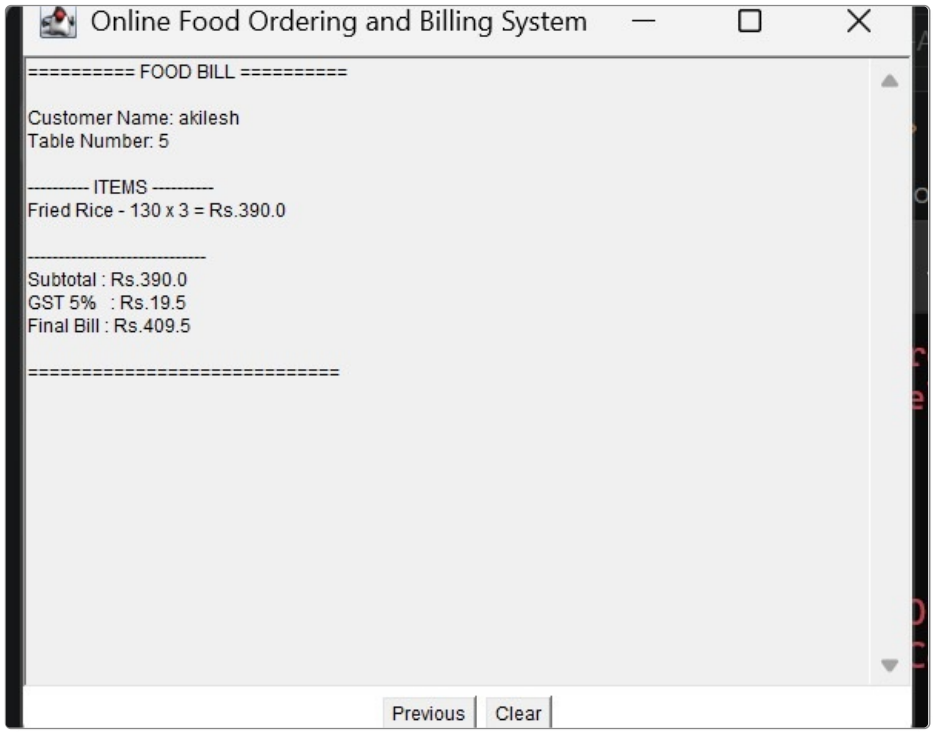
**Generated Bill:**

```
========= BILL SUMMARY =========
Customer : Rohit Sharma
Table No : 7
--------------------------------
Veg Biryani          x2  Rs.  440.00
Masala Chai          x3  Rs.  120.00
--------------------------------
Subtotal        : Rs.560.00
GST (5%)        : Rs.28.00
FINAL AMOUNT    : Rs.588.00
================================
```




*Task 2 — Card 1: Customer / Order Details screen*


*Task 2 — Card 2: Food Selection screen (item added to order)*




*Task 2 — Card 3: Bill / Order Summary screen*

## Compilation & Execution

```
javac FoodOrderingSystem.java
java FoodOrderingSystem
```

## Evaluation Rubric

| Criterion | Weightage | Marks / Remarks |
|---|---|---|
| Problem Understanding | 15% | |
| GUI Components and Layout | 20% | |

| | | |
|---|---|---|
| Event Handling | 20% | |
| Functional Correctness | 25% | |
| Input Validation and User Experience | 10% | |
| Program Explanation / Viva | 10% | |

## Student Instructions

1. The solution must be implemented in Java using the concepts covered in the laboratory syllabus.
2. Students should demonstrate compilation, execution, and output during the practical session.
3. Use meaningful variable names and appropriate comments.
4. Students must be able to explain the selected AWT components, layout manager, event handling mechanism, and processing logic.